计算机体系结构

08. 指令集架构

李建华
计算机与信息学院

The contents of the slides are adapted from CA course of wisc, princeton, mit, berkeley, edinburg, and eth. The uses of the slides of this course are for educational purposes only and should be used only in conjunction with the textbook. Derivatives of the slides must acknowledge the copyright notices of this and the originals.

本讲提纲

- **冯诺依曼模型**
  - **所有当代基于ISA的计算机的潜在模型**
- 指令格式
  - 长度 和 编码
- 操作类型
- 操作数模型
  - 操作数如何存储、如何访问?
- 数据类型及其操作
- 流程控制

《系统硬件综合设计》的设计目标：
基于LoongArch，MIPS，RISC-V，或自定义ISA，设计功能相对完善的单周期、多周期、流水线CPU。

冯·诺依曼模型

- 程序计数器是关键
  - 决定了动态处理的指
  - 下一条指令的PC**通**
  - 指令和数据均放在存
    义了计算，**构成了算**
    - 值从指令X传递到指
    - X将A作为输出，Y
    - Y在X之后处理

```
# Loop to compute each Fibonacci number using the previous two Fib. numbers.
loop: lw   $s3, 0($s0)      # Get value from array F[n-2]
      lw   $s4, 4($s0)      # Get value from array F[n-1]
      add  $s2, $s3, $s4    # F[n] = F[n-1] + F[n-2]
      sw   $s2, 8($s0)      # Store newly computed F[n] in array
      addi $s0, $s0, 4      # increment address to now-known Fib. number storage
      addi $s1, $s1, -1     # decrement loop counter
      bgtz $s1, loop        # repeat while not finished

# Fibonacci numbers are computed and stored in array. Print them.
      la   $a0, fibs        # first argument for print (array)
      add  $a1, $zero, $s5  # second argument for print (size)
      jal  print            # call print routine.

# The program is finished. Exit.
      li   $v0, 10          # system call for exit
      syscall               # Exit!
```

- 处理器在逻辑上一直处理右图中的循环
  - 假定指令的处理是原子性的
  - 假定指令X在指令X+1开始处理之前完成
    - 现代处理器不遵守上面的二条约定

本讲提纲

- 冯诺依曼模型
  - 所有当代基于ISA的计算机的潜在模型
- **指令格式**
  - **长度 和 编码 （以MIPS为例）**
- 操作类型
- 操作数模型
  - 操作数如何存储、如何访问?
- 数据类型及其操作
- 流程控制

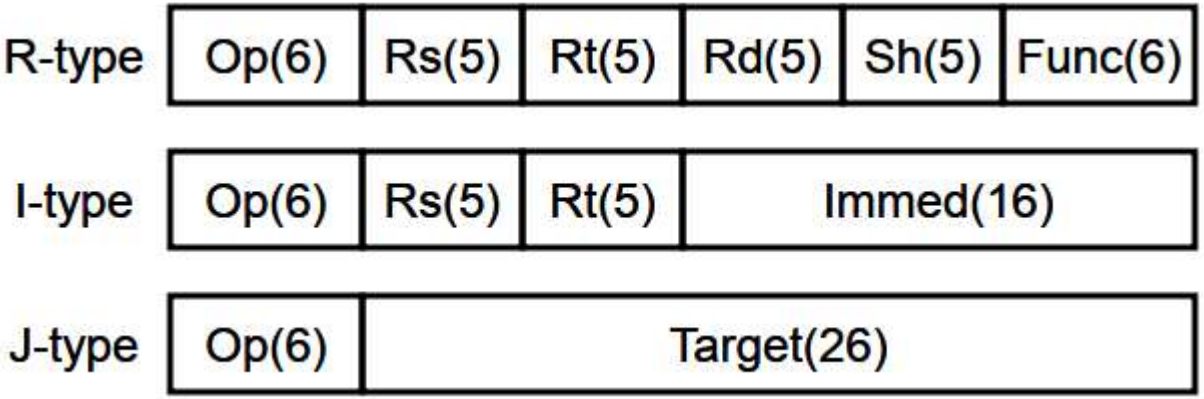
指令格式

- 基于编码长度的分类
  - 固定长度编码
    - 32 或者 64 位 （嵌入式系统里16位） **why?**
    - 优点是易于实现: 基于PC计算下一条指令地址很方便，且译码效率高。
    - 缺点：low code density
  - 可变长度编码
    - 优点：high code density
    - 缺点：实现复杂，complex decoding logic
  - 折中方案: 混合上述2种方案
    - 示例: MIPS16，16 and 32bit encodings

MIPS32指令格式

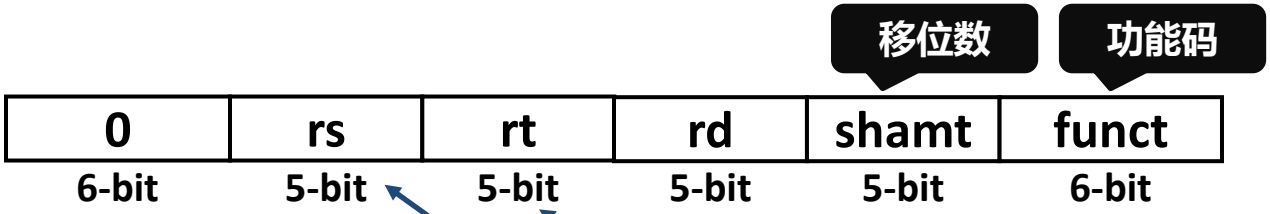
- 编码格式
  - 3种格式，编码简单
  - 问：6位的操作码能表示多少种不同的指令?

Microprocessor without Interlocked Pipeline Stages

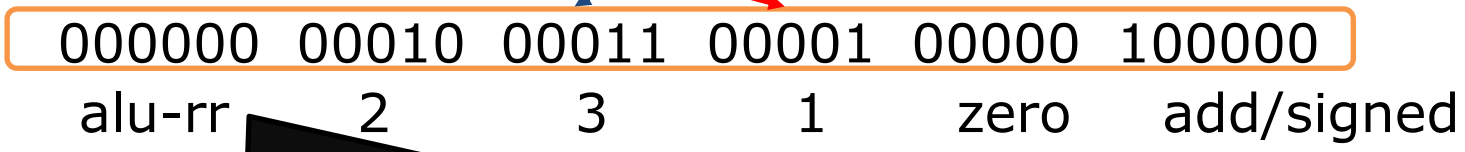


X64 ISA： http://ref.x86asm.net/geek64-abc.html

R-type (寄存器类型)



- 例如： add $1, $2, $3



- **CPU能直接处理的信息就是这样的二进制编码，指令和数据皆是如此；**
- **这些二进制信息存储在计算机的memory里面。**

MIPS指令的function域

MIPS的R-type指令的操作码是“000000”。Funct 定义了具体的功能。

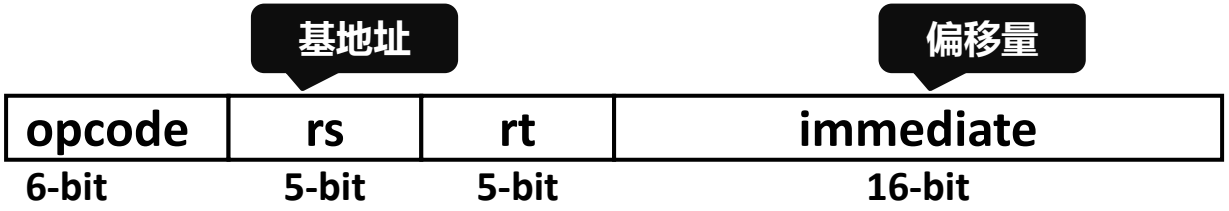
Table B.2 R-type instructions, sorted by funct field

Funct	Name	Description	Operation
000000 (0)	sll rd, rt, shamt	shift left logical	[rd] = [rt] << shamt
000010 (2)	srl rd, rt, shamt	shift right logical	[rd] = [rt] >> shamt
000011 (3)	sra rd, rt, shamt	shift right arithmetic	[rd] = [rt] >>> shamt
000100 (4)	sllv rd, rt, rs	shift left logical variable	[rd] = [rt] << $[rs]_{4:0}$
000110 (6)	srlv rd, rt, rs	shift right logical variable	[rd] = [rt] >> $[rs]_{4:0}$
000111 (7)	srav rd, rt, rs	shift right arithmetic variable	[rd] = [rt] >>> $[rs]_{4:0}$
001000 (8)	jr rs	jump register	PC = [rs]

Table B.2 R-type instructions, sorted by funct field—Cont'd

Funct	Name	Description	Operation
100000 (32)	add rd, rs, rt	add	[rd] = [rs] + [rt]
100001 (33)	addu rd, rs, rt	add unsigned	[rd] = [rs] + [rt]
100010 (34)	sub rd, rs, rt	subtract	[rd] = [rs] - [rt]
100011 (35)	subu rd, rs, rt	subtract unsigned	[rd] = [rs] - [rt]
100100 (36)	and rd, rs, rt	and	[rd] = [rs] & [rt]
100101 (37)	or rd, rs, rt	or	[rd] = [rs] | [rt]
100110 (38)	xor rd, rs, rt	xor	[rd] = [rs] ^ [rt]
100111 (39)	nor rd, rs, rt	nor	[rd] = ~([rs] | [rt])
101010 (42)	slt rd, rs, rt	set less than	[rs] < [rt] ? [rd] = 1 : [rd] = 0
101011 (43)	sltu rd, rs, rt	set less than unsigned	[rs] < [rt] ? [rd] = 1 : [rd] = 0

I-type (❶访存类指令)



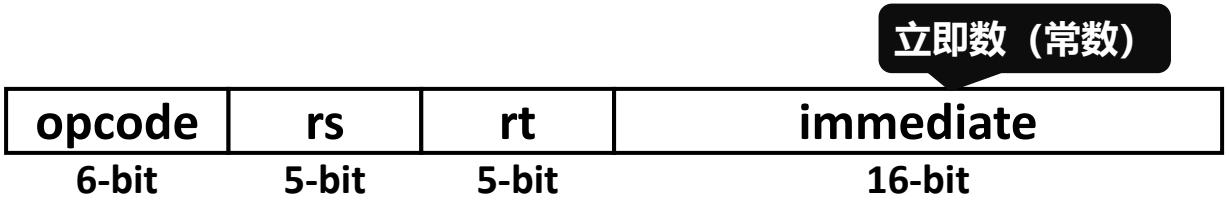
- 所有的loads 和 stores 类指令都是I-type
- 例如： lw $1, 100($2)

访存地址

100011 00010 00001 0000000001100100

lw 2 1 100 (in binary)

I-type (❷立即数类型 ❸条件分支)



- 带有立即数的算术运算指令，也是I-type
  - addi $1, $2, 100
  - 001000 00010 00001 0000000001100100
- 条件分支指令，也是I-type
  - beq $1, $2, 7
  - 000100 00001 00010 0000 0000 0000 0111
  - PC = **PC+4** + (0000 0111 << 2) // word offset

I-type有3个子类，大家有何发现?

J-type (跳转类型)



- 绝对跳转指令:
  - j target
  - opcode = 2
- 跳转的目标地址:
  - 目标地址 = (PC+4)[31:28] || target || 00

注意：这里，很多中文教材都误认为是PC!

本讲提纲

- 冯诺依曼模型
  - 所有当代基于ISA的计算机的潜在模型
- 指令格式
  - 长度 和 编码
- **操作类型**
- 操作数模型
  - 操作数如何存储、如何访问?
- 数据类型及其操作
- 流程控制

操作类型

A.M. TURING CENTENARY CELEBRATION WEBCAST

acm

MORE ACM AWARDS

A.M. TURING AWARD

A.M. TURING AWARD LAUREATES BY...

ALPHABETICAL LISTING | YEAR OF THE AWARD | RESEARCH SUBJECT



PHOTOGRAPHS

JOHN COCKE DL

United States – 1987

CITATION

For significant contributions in the design and theory of compilers, the architecture of large systems and the development of reduced instruction set computers (RISC); for discovering and systematizing many fundamental transformations now used in optimizing compilers including reduction of operator strength, elimination of common subexpressions, register allocation, constant propagation, and dead code elimination.

图灵奖获得者：John Cocke

本讲提纲

- 冯诺依曼模型
  - 所有当代基于ISA的计算机的潜在模型
- 指令格式
  - 长度 和 编码
- 操作类型
- **操作数模型**
  - **操作数如何存储、如何访问?**
- 数据类型及其操作
- 流程控制

操作数模型

- 思考：如果要做加法运算，至少需要3个操作数。
  - 2个源操作数和1个目标操作数
- 自然产生2个问题：
  - 问题1：操作数来自哪里?
  - 问题2：操作数如何编码?
- 下面以“A = B + C”为例讨论4种操作数模型：
  - Memory-Only
  - Accumulator
  - Stack
  - Registers

这就涉及操作数模型

Memory-Only

- Memory only

add A,B,C **mem[A] = mem[B] + mem[C]**



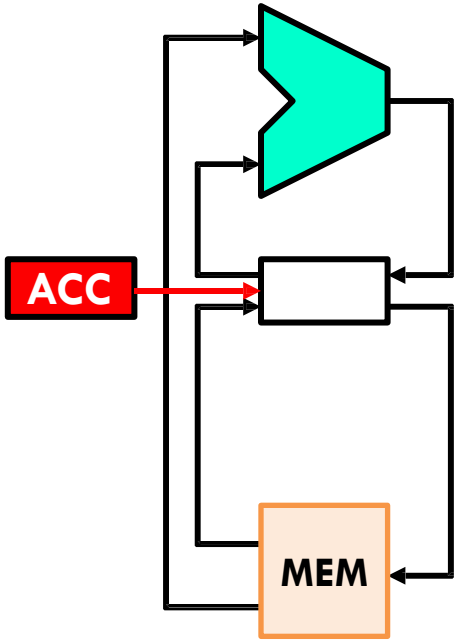
Accumulator

- Accumulator: 就是一个具有单个元素的stack

load B	**ACC = mem[B]**
add C	**ACC = ACC + mem[C]**
store A	**mem[A] = ACC**

The **accumulator** is the register for storing results of operations inside the CPU. Accumulator's name or representation depend upon CPU architecture and manufacturer, for example in intel x86 architecture accumulator's name is AX in 16bits or EAX in 32bits mode. An accumulator is manipulated by machine code or assembly for example assembly code:

"add ax, 3" add 3 in accumulator register.



https://simple.wikipedia.org/wiki/Accumulator

Stack

- Stack: top of stack (TOS) is implicit in instructions

push B	**stack[TOS++] = mem[B]**
push C	**stack[TOS++] = mem[C]**
add	**stack[TOS++] = stack[--TOS] + stack[--TOS]**
pop A	**mem[A] = stack[--TOS]**

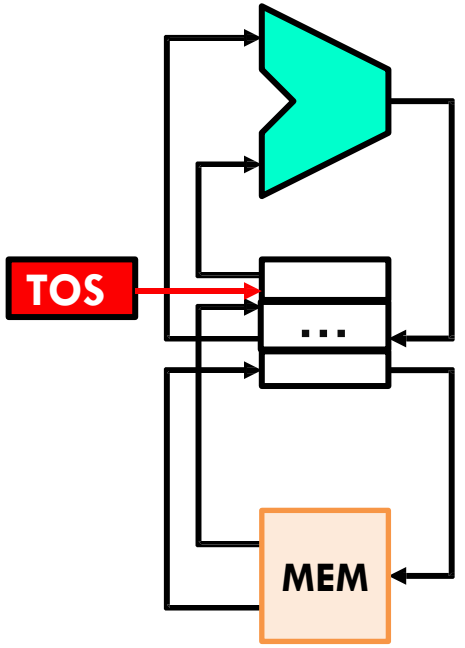
Stack registers in x86 [edit]

In 8086, the main stack register is called "stack pointer" (SP). The stack segment register (SS) is usually used to store information about the memory segment that stores the call stack of currently executed program. SP points to current stack top. By default, the stack grows downward in memory, so newer values are placed at lower memory addresses. To save a value to the stack, the `PUSH` instruction is used. To retrieve a value from the stack, the `POP` instruction is used.

Example: Assuming that SS = 1000h and SP = 0xF820. This means that current stack top is the physical address 0x1F820 (this is due to memory segmentation in 8086). The next two machine instructions of the program are:

```
PUSH AX
PUSH BX
```

https://en.wikipedia.org/wiki/Stack_register



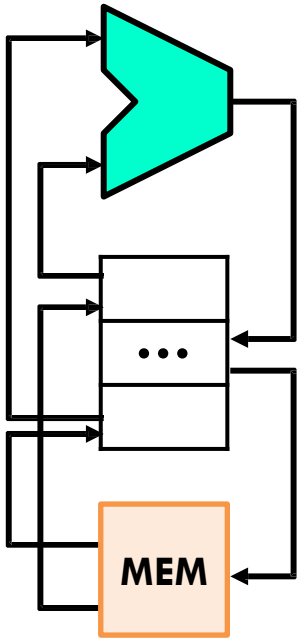
Register

- 通用寄存器：也就是多个显式的accumulators

```
load  R1, B      R1 = mem[B]
add   R1, C      R1 = R1 + mem[C]
store R1, A      mem[A] = R1
```

- Load-store: 基于GPR，且只有load/store指令可以访问存储器。

```
load R1, B       R1  =  mem[B]
load R2, C       R2  =  mem[C]
add  R1, R1, R2  R1 = R1 + R2
store R1, A      mem[A] = R1
```



3种操作数模型的对比

- 静态代码量 - static code size
  - 用来表示一个程序的指令的数量
  - Memory-only < register < load-store
- 数据存储器流量 - data memory traffic
  - 读写存储器的字节数 (数据量)
  - load-store < register < memory only
- 指令处理延迟 - instruction latency
  - 处理指令平均所需的时间
  - load-store < register < memory only

MIPS的操作数模型

- MIPS是load-store模型
  - 32个32位整型寄存器
    - 实际上只有31个，因为0号寄存器永远为0；
  - 32个32位浮点寄存器
    - 也可以作为16个64位浮点寄存器使用
  - HI, LO: multiply/divide的目标寄存器

MIPS整型寄存器用途

- 简化编译器设计
  - 函数级别的编译和函数调用分开处理

关于MIPS ISA更多的内容请参考MIPS Green Card和MIPS ISA手册

名称	编号	用途
$0	0	the constant value 0
$at	1	assembler temporary
$v0-$v1	2-3	function return value
$a0-$a3	4-7	function arguments
$t0-$t7	8-15	temporary variables
$s0-$s7	16-23	saved variables
$t8-$t9	24-25	temporary variables
$k0-$k1	26-27	OS temporaries
$gp	28	global pointer
$sp	29	stack pointer
$fp	30	frame pointer
$ra	31	function return address

寻址模式

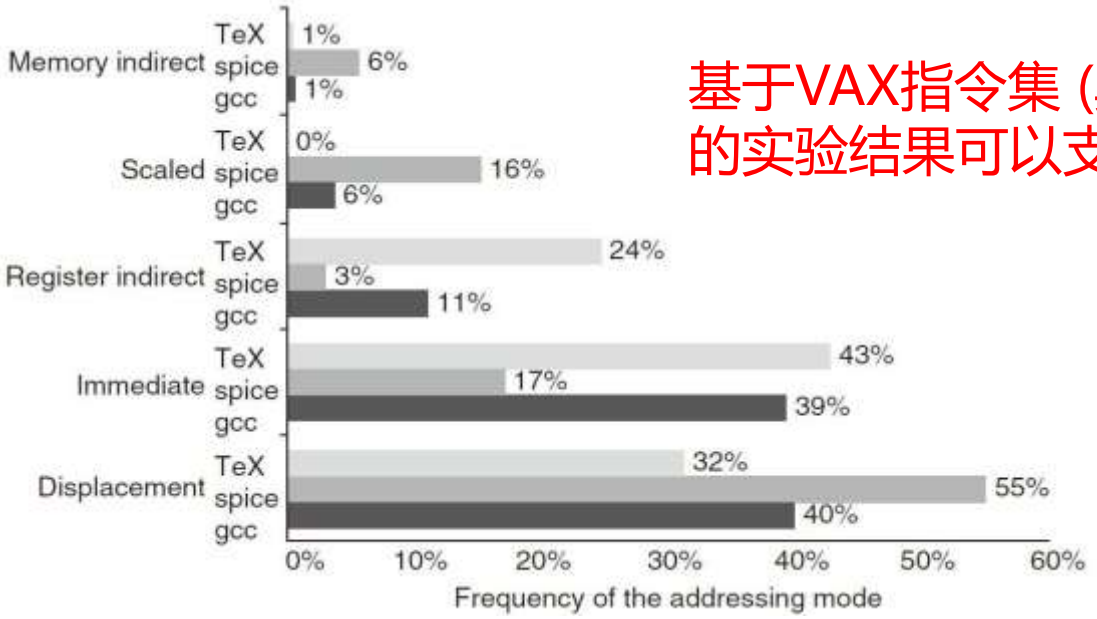
- 现代计算机系统通常使用虚拟地址模式
  - 典型的是32位或者64位地址
  - 程序可以使用2^{32} 字节 (4GB) or 2^{64} 字节 (16PB)
  - 从ISA编码的角度：甚至没有空间来存放一个虚拟地址
- 寻址模式：就是用来处理地址编码问题的

Direct: ld R1, (R2)	R1=mem[R2]
Displacement: ld R1, 8(R2)	R1=mem[R2+8]
Indexed: ld R1, (R2,R3)	R1=mem[R2+R3]
Memory-indirect: ld R1, @(R2)	R1=mem[mem[R2]]
Scaled: ld R1,(R2,R3,32,8)	R2+=8; R1=mem[R2+R3*32+8]
…	…

X86 uses 10+ addressing modes.

MIPS的寻址模式

- MIPS只实现了偏移寻址 (displacement)
  - Disp: 61%, reg-ind: 19%, scaled: 11%, mem-ind: 5%, other: 4%
  - 80% 用偏移寻址或者寄存器间接寻址(偏移为0)



基于VAX指令集 (具有所有寻址模式)的实验结果可以支撑

How about the remain 20%?

本讲提纲

- 冯诺依曼模型
  - 所有当代基于ISA的计算机的潜在模型
- 指令格式
  - 长度 和 编码
- 操作类型
- 操作数模型
  - 操作数如何存储、如何访问?
- **数据类型及其操作**
- 流程控制

数据类型

- 数据类型
  - 软件的视角: 指的是数据的特性，可以进行各种抽象，例如数据结构里面各种数据类型；
  - 硬件的视角: 数据就是比特, 是操作的特性。仅支持基本的数据类型。
- 硬件数据类型
  - 整型: 8 bits (byte), 16b (half), 32b (word), 64b (long)
  - IEEE754 浮点型: 32b (单精度), 64b (双精度)
  - 打包的整型: 将 64b int 看成 8个 8b int 或者看成 4个 16b int型

MIPS数据类型

- MIPS ISA支持针对基础类型的各粒度的访存操作
  - 所有整型操作，32位的读写
  - 设置基于字节/半字（有符号/无符号）的load/store
    - lb, lbu, lh, lhu, sb, sh，…
- 操作: 对基本数据类型也有对应的运算操作
  - 整型算术指令支持有符号和无符号类型
  - 上述指令还只有立即数的变体
    - add, addu, addi, addiu，…
- 主要设计原则：Regularity/orthogonality
  - 所有的操作具有相应的变体指令类型
  - 方便compiler的工作，任意组合指令
  - RISC架构的性能很大程度依赖compiler

本讲提纲

- 冯诺依曼模型
  - 所有当代基于ISA的计算机的潜在模型
- 指令格式
  - 长度 和 编码
- 操作类型
- 操作数模型
  - 操作数如何存储、如何访问?
- 数据类型及其操作
- **流程控制**

流程控制①：分支条件判断

- 分支条件的测试
  - 方法1: 比较并分支 指令

```
blti $1,10,target
```

    - \+ 简单；– 二个ALUs: 一个判断条件, 一个计算目标；

  - 方法2: 隐式的条件码

```
subi $2,$1,10      // sets "negative" CC
bn target
```

    - \+ 条件码设置是“for free”；– 隐式依赖难以处理；

  - 方法3：条件寄存器:

```
slti $2,$1,10      // $2是条件寄存器
bnez $2,target
```

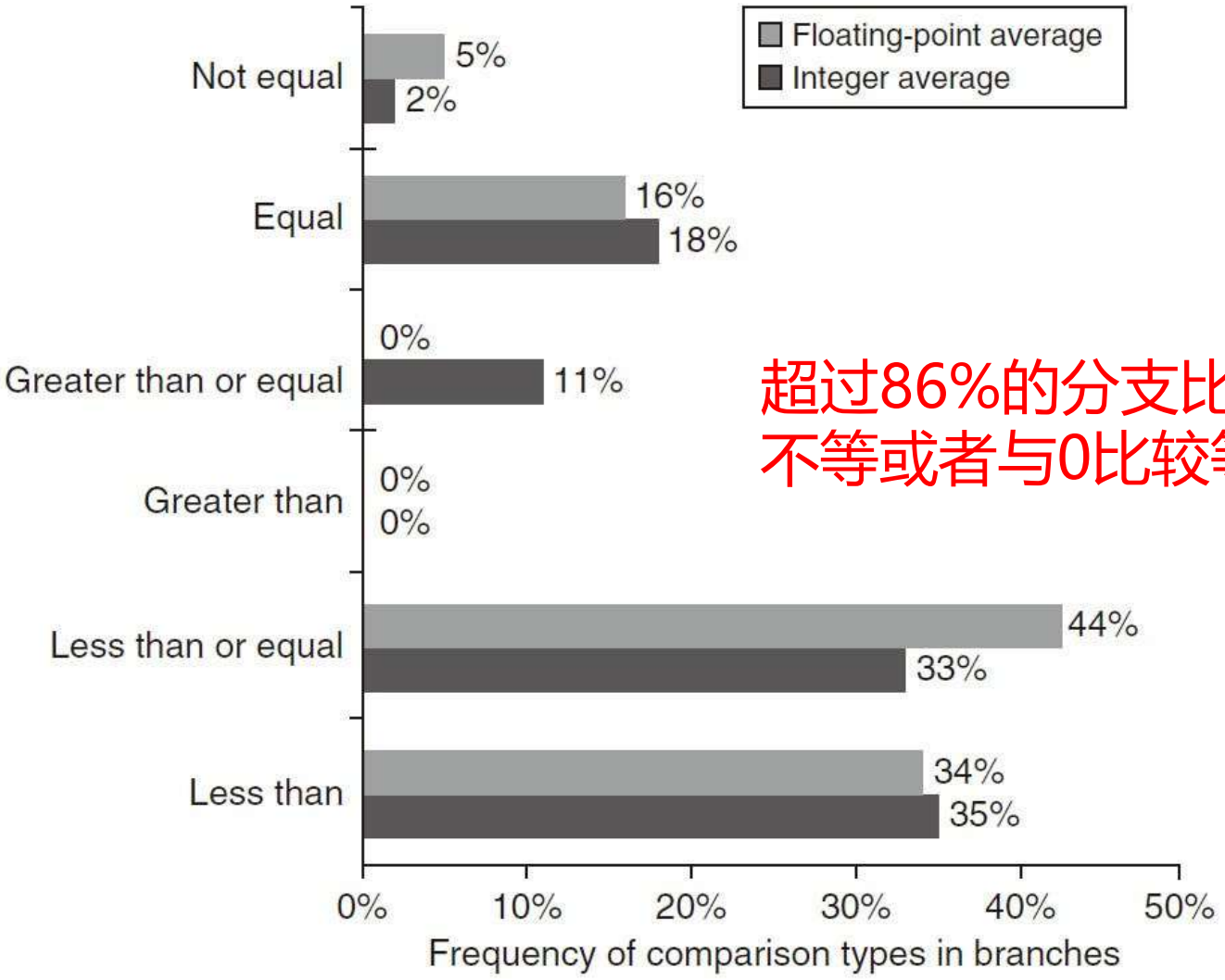
    - \+ one ALU per, + 显式的依赖关系； – 额外的分支指令；

MIPS的实现方式

- MIPS 使用方法1和3来实现流程控制
  - 比较并分支指令 (方法1): beq, bne
    - 只进行相等和不等的比较
    - 不需要专门的ALU或加法器 +
  - 将寄存器与0比较进行分支跳转 (方法1): bgtz, bgez, bltz, blez
    - 只进行大于或者小于比较
    - 不需要专门的ALU或加法器
  - 设置显式的条件寄存器 (方法3): slt, sltu, slti, sltiu, etc.

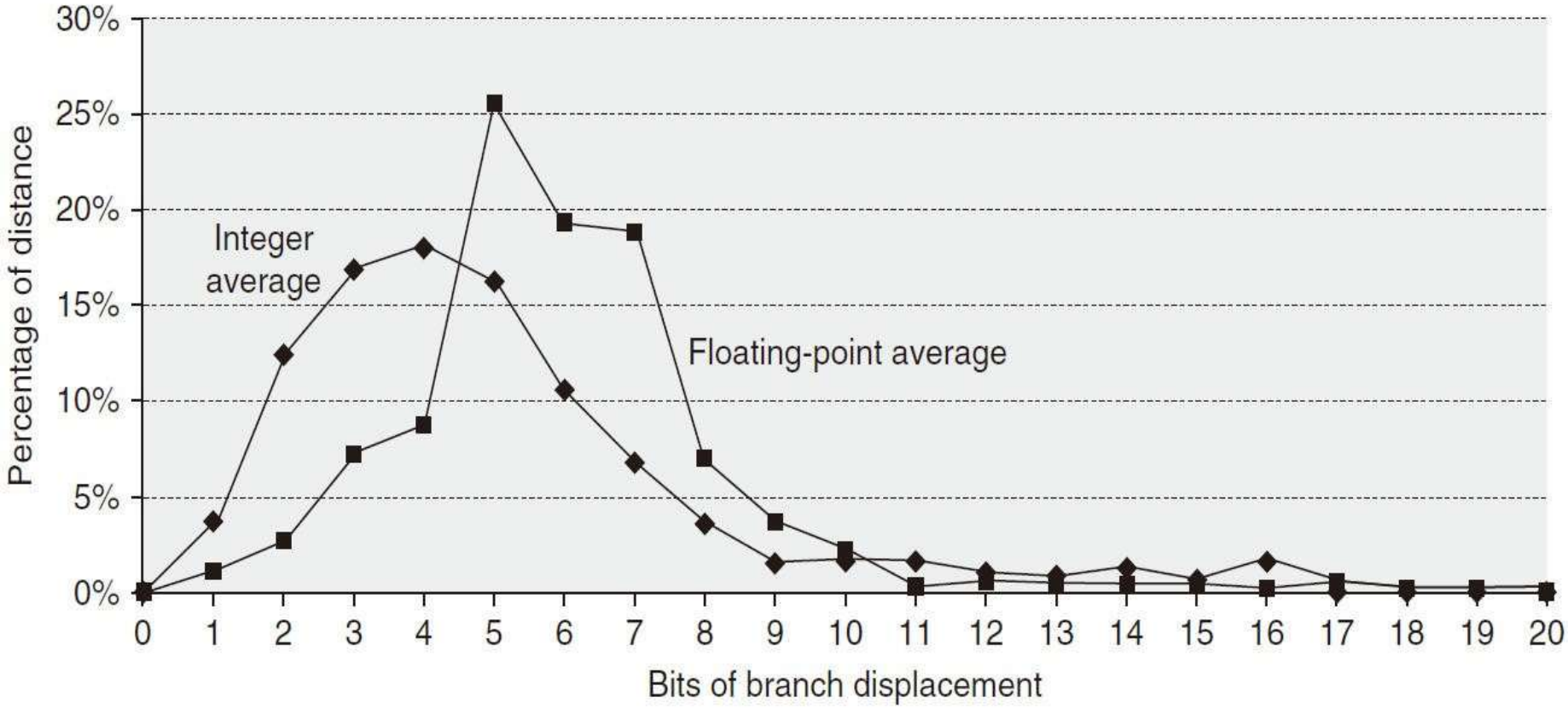
- 为什么MIPS这样选择?

分支条件判断的使用情况



超过86%的分支比较是相等、不等或者与0比较等简单形式。

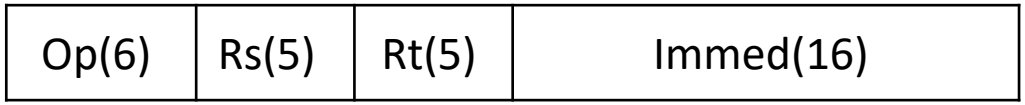
流程控制②：跳多远



- 通常会跳多远？等价于：需要多少位偏移量？
  - 对于过程内部的跳转来说，应该不会很远。Why?
  - 从一个过程跳到另外一个过程则可能很远。Why?

MIPS的实现方式

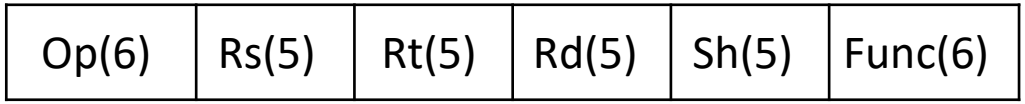
- MIPS使用了3种方法
  - PC-relative: bne, beq, blez, etc.
    - 16位偏移量, <0.1% 需要更多位的偏移量
    - 若条件为真，PC = PC+4+immediate，否则：PC=PC+4



  - Absolute: j target
    - 26位偏移，可寻址2^{28} words < 2^{32}



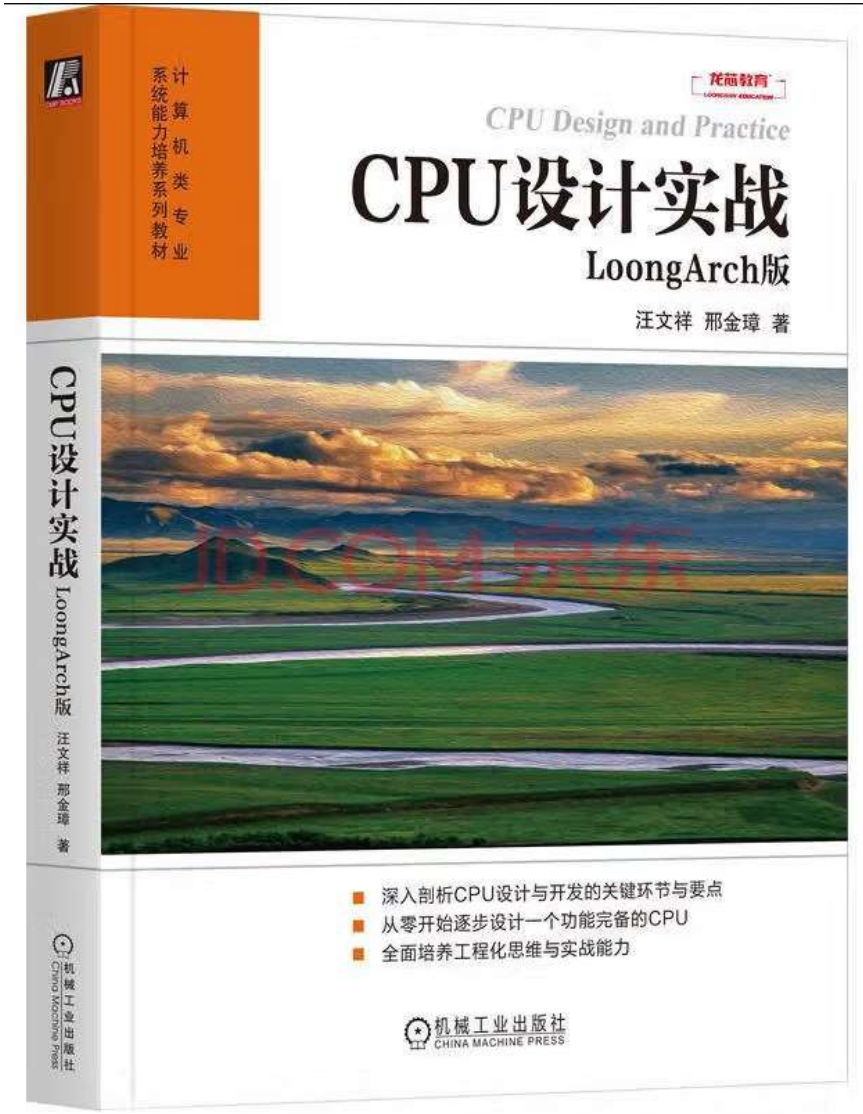
  - Indirect: jr $rd
    - 32位地址，足够了。

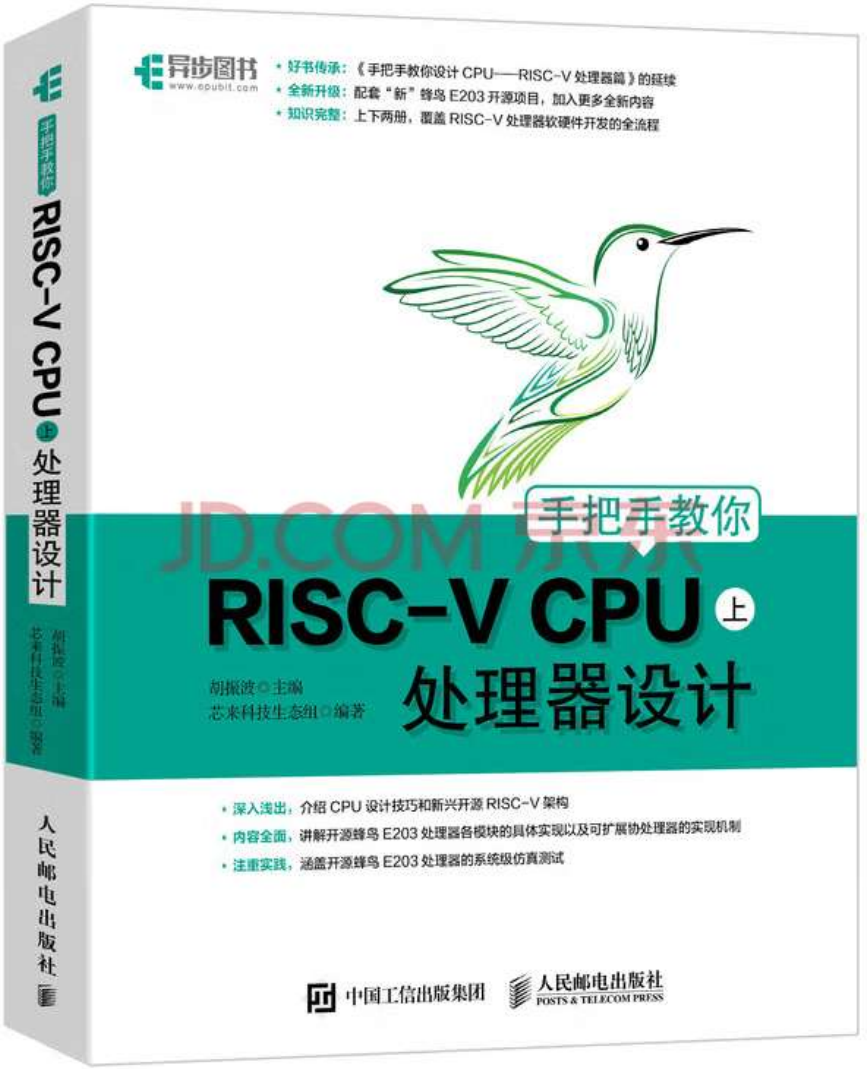


流程控制③：过程调用

- 如何支持过程调用?
  - 记住调用指令的地址 (current PC + 4)，这样调用结束之后就可以返回。
- MIPS实现方式
  - 隐式的返回地址寄存器：$ra(=$31)
  - 直接的jump-and-link指令: jal address
    - $ra = PC+4; PC = address
    - 可以通过jr指令返回: jr $ra
  - 或者可以使用间接的jump-and-link指令: jalr $rd, $rs
    - $rd = PC+4; PC = $rs// explicit return address register
    - 然后使用jr指令返回: jr $rd

《系统硬件综合设计》参考教材

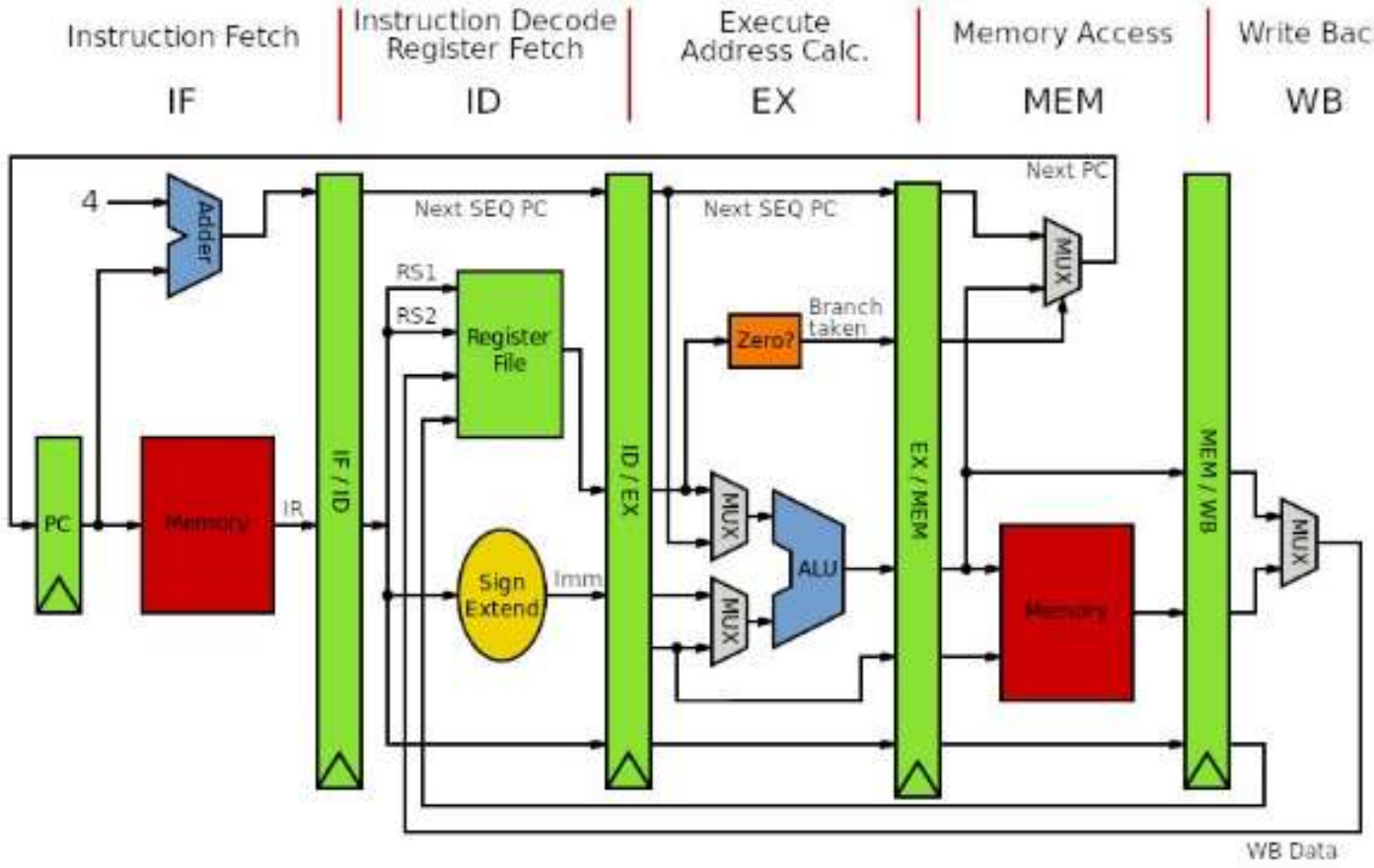




https://bookdown.org/loongson/_book3/

图片来源于jd.com

Next Topic


Instruction Fetch
IF
Instruction Decode
Register Fetch
ID
Execute
Address Calc.
EX
Memory Access
MEM
Write Back
WB
4
Adder
Next SEQ PC
Next SEQ PC
Next PC
MUX
RS1
RS2
Register
File
Zero?
Branch
taken
PC
Memory
IR
IF / ID
ID / EX
EX / MEM
MEM / WB
MUX
ALU
Sign
Extend
Imm
Memory
MUX
WB Data